SISTEMAS DISTRIBUÍDOS

Ronaldo Cesar Mengato Junior



SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

Gestão da replicação

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Descrever os modelos de sistema e o papel da replicação.
- Examinar exemplos práticos de replicação e estratégias.
- Ilustrar aplicações de modelos de replicação.

Introdução

A replicação é fundamental nos sistemas distribuídos modernos, seja para a melhoria de desempenho, aumento da disponibilidade de serviços ou tolerância a falhas. Além da replicação de dados, é comum, atualmente, replicar rotinas que executam pequenas tarefas, chamadas de microsserviços.

Apesar de trazer algumas vantagens para a gestão, manter réplicas em diversos computadores pode ser uma estratégia bastante complexa e difícil para manter a consistência dos dados replicados. Há aplicações que precisam escolher entre alta disponibilidade ou rígida política de consistência.

Neste capítulo, você conhecerá o papel da replicação e as dificuldades inerentes a ela, além das estratégias e modelos utilizados na gestão de dados replicados em diferentes computadores. Por fim, verá alguns exemplos de aplicações reais, para exemplificar instâncias concretas dos componentes discutidos nos modelos.

1 Modelos de sistemas e função da replicação

O fato de podermos manter cópias de informações em vários computadores habilita o uso de sistemas mais complexos e robustos, o que torna a replicação bastante utilizada atualmente. Por exemplo, se você acessou a internet hoje, é provável que tenha acessado dados no cache do seu navegador, ou seja,

a informação do *site* que você buscou estava salva em uma cópia no seu próprio computador. Veja as principais vantagens de se utilizar a replicação a seguir.

- Melhoria do desempenho da aplicação: o exemplo do cache de dados pelo navegador é uma melhoria no tempo de resposta da aplicação, pois o cliente não precisa acessar o servidor remoto que possui uma latência de rede maior. Essa melhoria não se limita apenas ao cache do navegador, mas qualquer aplicação pode utilizar essa técnica.
- Maior disponibilidade: talvez um dos principais motivos para usarmos dados replicados. Atualmente, é inaceitável esperar para obter dados, por exemplo, os de redes sociais, que devem ser entregues imediatamente, ou perderão usuários. Por isso, a maioria dos serviços possuem uma taxa de disponibilidade próxima dos 100%, ou seja, a aplicação fica disponível 24 horas por dia, 7 dias por semana, o que só é possível se os dados forem replicados.
- Tolerância a falhas: alguns serviços precisam de um comportamento rigorosamente correto, mesmo que haja falhas durante a execução, o objetivo deve ser alcançado. O fato de existirem dados de alta disponibilidade não significa, necessariamente, que são corretos ou atualizados. Imagine o caso de uma rede social em que um usuário troca a sua foto de perfil, essa informação não tem necessidade de ser replicada imediatamente para todos os outros usuários; porém, isso não pode ocorrer em sistemas críticos, como um controle de tráfego aéreo, no qual os dados corretos são necessários em escalas de tempo curtas.

Apesar de ser um processo complexo, que pode envolver diversas estratégias, o fato de uma aplicação possuir dados replicados deve ser transparente ao usuário, ou seja, ele não precisa saber que existem várias cópias físicas dos dados. Além disso, em geral, os usuários esperam receber apenas um determinado conjunto de dados, mesmo que o serviço opere com mais, devendo retornar somente aquilo que foi solicitado (COULOURIS *et al*., 2013).

Outro exemplo de requisito para dados replicados é a consistência, que pode variar de acordo com o tipo da aplicação, algumas mais rígidas outras mais flexíveis. As operações de replicação em diferentes computadores devem produzir resultados que satisfaçam os requisitos da aplicação. Por exemplo, um sistema bancário que deve replicar, para os outros bancos, uma operação de crédito e outra de débito, exatamente nessa ordem, pode apresentar um valor de saldo da conta diferente e causar problemas ao usuário, se, por acaso, algum dos servidores inverterem as operações.

Segundo Tanenbaum e Steen (2008), manter várias cópias consistentes pode causar problemas de escalabilidade. Portanto, compreende-se como consistência, quando qualquer operação de consulta em qualquer réplica de dados retorna o mesmo conjunto de informações. Por isso, muitas vezes, as aplicações precisam aumentar a escalabilidade (a capacidade de crescer rapidamente) ou manter um esquema de dados consistente.

A Figura 1 mostra um modelo de arquitetura básica de gestão da replicação, definido por Coulouris *et al*. (2013), no qual as réplicas dos dados são mantidas pelos "gerenciadores de réplica", que (usando a arquitetura cliente-servidor) podem ser qualquer dispositivo que esteja servindo aos dados replicados. Portanto, podem ser chamados simplesmente de servidores.

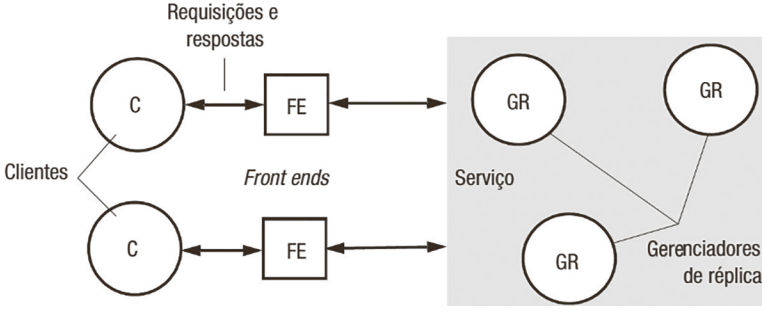


Figura 1. Modelo de arquitetura básica para gestão da replicação.

Fonte: Coulouris *et al*. (2013, p. 769).

As operações que um gerenciador executa em suas réplicas devem ser recuperáveis e atômicas (indivisíveis), ou seja, a operação não deve deixar a réplica inconsistente. Caso haja falha durante o processo, deve ser possível desfazer. Essas características, à medida que se tem mais servidores replicados, traz complexidade para a aplicação, principalmente considerando sistemas tolerantes a falhas. É preciso considerar o consenso de tempo entre as máquinas e os tempos das mensagens de sincronização.

No modelo especificado na Figura 1, os clientes fazem requisições de serviços aos servidores, e essas chamadas podem envolver operações de leitura de dados ou de atualização (quando há escrita ou alteração de um objeto ou arquivo). Antes de chegar ao serviço de destino, a requisição passa por um componente chamado *front-end*, cuja função é se comunicar com um ou mais gerenciadores de réplica, em vez de obrigar o cliente a fazer isso, tornando o processo transparente ao usuário (COULOURIS *et al*., 2013).

É importante entender o funcionamento do modelo quando uma requisição é disparada, para isso, veja a seguir as cinco fases que, segundo Coulouris *et al*. (2013), podem afetar o desempenho.

1. Requisição: é disparada pelo *front-end* para um ou mais gerenciadores. Caso seja para apenas um gerenciador, ele pode se comunicar com outros nós, contudo, o *front-end* também pode fazer um *multicast* para todos os servidores e aguardar a resposta de alguns.
2. Coordenação: ocorre quando os gerenciadores verificam se a requisição deve ser aplicada e qual a sua ordem, caso haja outras esperando. Podem ser adotados diferentes tipos de ordenação, como *first-in first-out* (FIFO), ordem causal ou ordem total.
3. Execução: a requisição executada de forma que possa ser desfeita em caso de falhas posteriores. Dependendo da implementação, a execução pode ocorrer em apenas um servidor ou em todos, caso o *front-end* tenha feito um *multicast*.
4. Acordo: nessa fase, os gerenciadores entram em um consenso sobre a requisição (se ela será aceita ou cancelada).
5. Resposta: um ou mais gerenciadores respondem ao *front-end*. Dependendo da implementação do *front-end*, ele pode pegar a primeira resposta e descartar as outras, ou aguardar a chegada de todas e, então, decidir.

Algumas aplicações podem não executar todas essas fases ou diversificar a ordem de cada uma delas. Imagine um aplicativo de *smartphone* que funcione com dados salvos localmente para quando o usuário estiver sem conexão com a internet. Nesse caso, quando surgir uma requisição, o *front-end* não poderá comunicar outros gerenciadores e deverá "pular" para a execução e uma resposta o mais rápido possível. Para manter a consistência, quando o usuário estiver conectado novamente, deverá ser replicada a requisição para outros servidores.

2 Estratégias de replicação

Nesta seção, serão abordadas duas estratégias de replicação: a passiva (*backup*), que é a mais comum, e a ativa.

No modelo de replicação passiva há a presença de um servidor primário e outros secundários (também chamados de *master* e *slaves*). A comunicação do *front-end* ocorre apenas com o gerenciador primário, em seguida, ele replica as operações realizadas para os servidores secundários. Caso o servidor primário falhe, um novo deve ser eleito como primário.

A Figura 2 exemplifica a troca de mensagens entre os componentes no modelo passivo de replicação, note que as setas saindo dos componentes de *front-end* vão apenas para um único gerenciador de réplica. Nesse cenário, a sequência de eventos quando um cliente solicita uma operação é a seguinte (COULOURIS *et al*., 2013):

- requisição — emitida pelo *front-end* para o gerenciador primário, podendo ter um identificador único para facilitar a sua coordenação;
- coordenação — o gerenciador verifica se a requisição já foi feita, devido ao fato de ela conter o seu identificador. Para que isso seja possível é necessário que o servidor mantenha um registro histórico das requisições atendidas;
- execução — a requisição é executada pelo servidor primário, e sua resposta deve ser armazenada, pois deve-se garantir a atomicidade e a capacidade de replicar essa operação;
- acordo — caso seja uma operação de atualização, o gerenciador deve emitir uma mensagem para que os outros também a executem. Os nós secundários devem enviar uma confirmação;
- resposta: o gerenciador primário envia a resposta ao *front-end* que retornará ao cliente final.

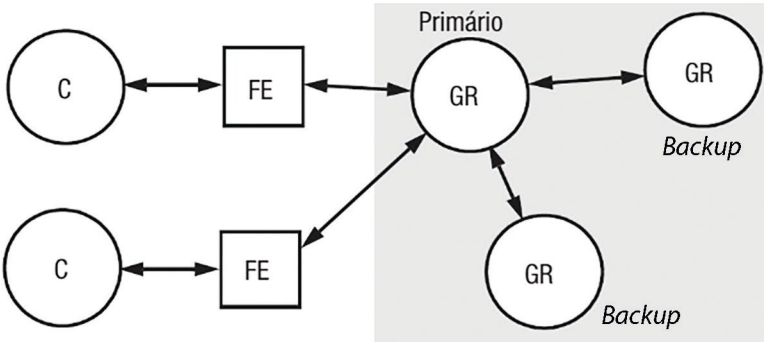


Figura 2. Modelo de replicação passiva (*backup* primário).
Fonte: Adaptada de Coulouris *et al.* (2013).

Olhando para o fluxo de operações realizado a cada requisição, é possível notar um problema que pode impactar a aplicação. Caso o sistema seja síncrono, ou seja, o cliente fique aguardando uma resposta, a etapa acordo pode causar lentidão, dependendo do número de gerenciadores secundários dentro do arranjo.

Outra preocupação com esse modelo é o caso de falha do gerenciador primário, em que um dos gerenciadores de *backup* deve assumir como primário, se mais de um assumir, causará falha no sistema. Os *backups* remanescentes devem concordar com o novo e, ainda, concordar com as operações que já foram realizadas, por isso é importante a replicação do histórico de execuções.

Para que o modelo de *backup* primário tolere até "F" falhas, é necessário F + 1 gerenciadores de réplica, o que pode ser uma implantação bastante cara, dependendo da aplicação. Além disso, esse modelo não pode tolerar falhas bizantinas. Em contrapartida, o *front-end* pode facilmente tolerar falhas, pois, uma vez que detecta uma, basta pesquisar pelo novo endereço do servidor primário.

Outra desvantagem desse modelo é a sobrecarga no gerenciador primário, pois todas as requisições chegam a ele primeiro. Uma variação que pode auxiliar esse problema é habilitar requisições de consultas, que não alteram dados e podem ser atendidas pelos gerenciadores de *backup*.

Saiba mais

Falhas bizantinas são aquelas em que os gerenciadores de réplicas não conseguem concordar com uma estratégia para falhas catastróficas no processo de replicação.

No modelo de replicação ativa, apresentado na Figura 3, os gerenciadores são organizados em grupos. O *front-end*, em vez de enviar suas requisições para apenas um servidor, envia para todos do grupo via *multicast*. Todos os nós processam a requisição de forma independente e enviam uma resposta ao *front-end*. Com isso, caso alguns deles falhem, não há impacto na operação e só haverá erro em caso de colapso geral.

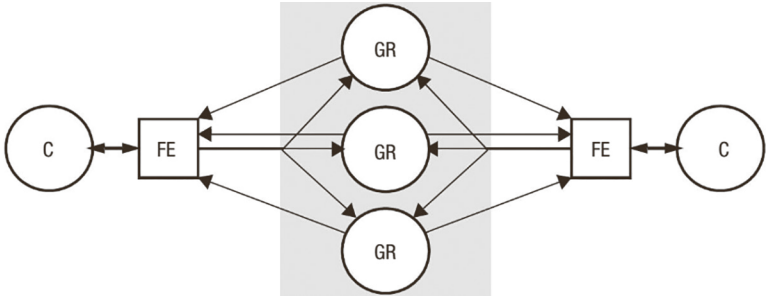


Figura 3. Modelo de replicação ativa.
Fonte: Coulouris *et al.* (2013, p. 781).

A execução de uma operação no modelo ativo, segundo Coulouris *et al.* (2013), segue as seguintes etapas:

- requisição — o *front-end* envia, via *multicast* totalmente ordenado e confiável, a requisição para todos os gerenciadores, garantindo que todos os receptores receberão a sua mensagem;
- coordenação — um sistema de comunicação em grupo envia a requisição para cada gerenciador de réplica na mesma ordem;

- execução — todos os nós executam a mesma requisição de forma similar, com isso, ao final da execução, todos devem estar no mesmo estado, garantindo a consistência;
- acordo — como a distribuição da requisição é total, não há necessidade de acordo entre os gerenciadores;
- resposta — cada gerenciador deve enviar a sua resposta ao *front-end* com o identificador da requisição; evidentemente, aqueles servidores que falharam durante a execução não enviam resposta alguma.

Considerando o modelo de resposta da replicação ativa, o conceito de falha pode ser relativo à aplicação, pois é possível definir que seja considerada falha apenas se um dos gerenciadores não responder, ou apenas se todos (ou uma boa parte) não enviarem a resposta. Com isso, a implementação do *front-end* também pode variar, sendo a política de aceitar a primeira resposta e descartar as outras adotada para um ganho de *performance*.

O sucesso do modo ativo de replicação depende bastante de um algoritmo de *multicast* totalmente ordenado e confiável para a entrega das requisições. Com isso e, ao menos, 2F + 1 de gerenciadores de réplicas, o modelo pode suportar até F falhas bizantinas. A fim de garantir de forma mais rígida a tolerância a falhas, pode-se adotar a prática de as respostas serem assinadas digitalmente pelos servidores.

3 Aplicações dos modelos de replicação

Nesta seção, serão apresentados dois exemplos de aplicações que utilizam, de algum modo, os conceitos de replicação. Com isso, será possível identificar casos reais e concretos e entender a importância do tema em sistemas distribuídos. No primeiro caso, será abordado um exemplo simples de replicação, utilizando o conceito de microsserviço, bastante popular atualmente. O segundo caso é um exemplo de implementação mais complexa da replicação em uma arquitetura desenvolvida por Ladin *et al*. (1992), que ficou conhecida pelo nome de *gossip* (fofoca).

Um microsserviço pode ser entendido como um projeto completo, porém, possui apenas uma funcionalidade. Por exemplo, um sistema que precise enviar um *e-mail* para seus clientes quando um evento ocorrer. Essa rotina pode ser escrita em um projeto a parte e ser usada em diferentes momentos na aplicação, além disso, esse serviço pode ser usado em outros projetos. Essa reusabilidade, aliada à fácil manutenção, tornou esse conceito bastante popular.

A Figura 4 apresenta um modelo para a replicação do serviço enviar *e-mails*. Perceba que, nesse caso, não há interesse em replicar dados, pois as informações necessárias para que o envio de *e-mail* ocorra podem ser passadas na própria requisição do *front-end*. Com isso, o modelo não precisa se preocupar em manter a consistência de dados, evitando uma grande complexidade.

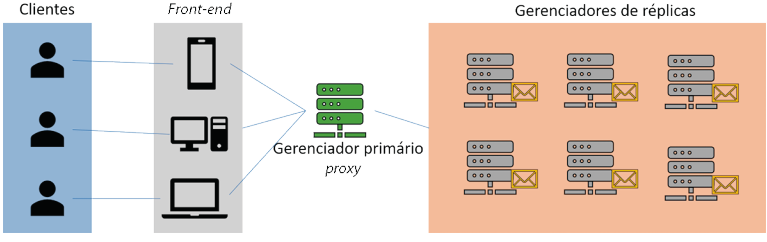


Figura 4. Modelo de replicação de serviços.

Na Figura 4 é possível reparar diferentes implementações de *front-end*. São exemplos válidos aplicativos para dispositivos móveis, *desktops*, *tablets*, *notebooks* etc. Quando um cliente deseja enviar um *e-mail*, o *front-end* manda uma requisição ao gerenciador primário, também chamado de *proxy*, que a direciona para algum gerenciador de réplica, nesse caso as instâncias do microsserviço enviar *e-mail*.

O gerenciador que atua como *proxy* possui políticas para balancear as requisições entre os servidores e nenhum fique sobrecarregado. Essa abordagem proporciona um melhor desempenho para a aplicação, pois o *proxy*, ao identificar que um servidor não retornou uma mensagem de êxito, pode atribuir a requisição para outro, e assim por diante. Se consideramos que a natureza de um envio de *e-mail* é assíncrona, não precisamos manter os clientes bloqueados enquanto as requisições são trocadas, o que traz transparência para o modelo.

O modelo de replicação de microsserviço apresentado é uma implementação do modelo passivo, mas com a exceção de não precisar manter um conjunto de dados. No caso de o *proxy* falhar, outro gerenciador pode ser eleito e exercer essa função. É importante lembrar que esses gerenciadores podem ser máquinas virtuais (ou containers), facilmente ligados ou desligados.

Outro exemplo de aplicação para a replicação é a arquitetura *gossip* explicada por Coulouris *et al*. (2013). Esse nome se refere ao fato de que os nós gerenciadores trocam mensagens de "fofoca" de tempos em tempos, a fim de transmitir as atualizações nas réplicas. Essa arquitetura é uma implementação de conceitos como replicação, consistência e sincronização. Pode ser usada por diferentes tipos de aplicações.

Os serviços da arquitetura *gossip* fornecem dois tipos de operações: consultas e atualizações, sendo que apenas a segunda modifica o estado da réplica. Os *front-ends* enviam requisições para os gerenciadores que escolherem, não necessariamente para todos ou para um primário, o que habilita a escolha de um que esteja disponível para responder em tempo razoável.

A Figura 5 apresenta as operações de consultas e atualizações em uma arquitetura *gossip*. É possível notar que, quando há uma consulta, o cliente aguarda uma resposta informando o estado da réplica, ao passo que na atualização nada é retornado. Outra característica é que cada cliente tem o seu próprio serviço, ou seja, o próprio gerenciador de réplicas ou um *cluster*. Essa característica de aplicações separa seus clientes via geolocalização, pois, assim, diminuem a latência.

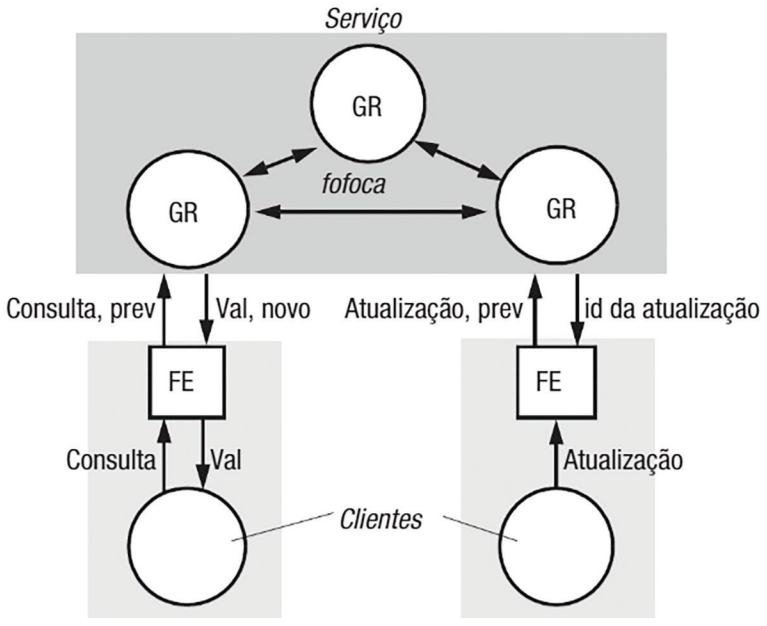


Figura 5. Operações de consulta e atualização em uma arquitetura *gossip*.
Fonte: Coulouris *et al*. (2013, p. 783).

Coulouris *et al*. (2013) explicam, de forma geral, que esse tipo de serviço processa as operações de consulta e atualização da seguinte forma:

- requisição — apesar de um *front-end* se comunicar com apenas um servidor, quando ele falha, procura outro que possa atender à requisição. Nas operações de atualização, o processo é retornado ao cliente assim que a operação tiver sido entregue ao *front-end*, então, ela é encaminhada ao servidor em segundo plano;
- resposta de atualização — o gerenciador responde assim que recebe a operação;
- coordenação — as mensagens de fofoca são a única forma de coordenação desse modelo, por isso um gerenciador não pode processar uma requisição até que possa aplicá-la, ou seja, pode ser que outros nós precisem receber atualizações antes. Esse comportamento é garantido pela ordenação total do modelo;
- execução — o gerenciador executa a requisição;

- resposta de consulta — essa fase é apenas para operações de consulta que exigem uma resposta ao cliente;
- acordo — os gerenciadores se atualizam por meio das mensagens de fofoca. Essa fase é executada de forma "preguiçosa", pois não precisa ocorrer ao término de cada requisição, e os nós são atualizados ocasionalmente, ou quando um servidor percebe que está desatualizado.

A implementação da arquitetura, em geral, fornece uma forma de consistência mais flexível, conforme você pode perceber pela sua fase de acordo. Nesse caso, considera-se que é uma consistência causal, e pode ser que algum cliente acesse dados antigos em relação a outros clientes. Outro fator que corrobora a condição de consistência relaxada é que essa arquitetura não determina um modo de propagação de atualizações, o que pode variar de acordo com a aplicação.

Uma rede social representa um cenário em que não é necessária uma consistência em tempo real e os serviços de fofoca podem ser aplicados. Por exemplo, imagine que um usuário da aplicação mora em algum país da Europa e possui amigos que também moram na mesma região e acompanham as suas publicações. O *cluster* no qual estão replicados os dados desse usuário também está localizado na Europa, assim como os dados de seus amigos. Porém, se um usuário da mesma rede social, mas que more no Brasil, tentar visualizar o perfil do usuário europeu poderá ver informações desatualizadas, pois o gerenciador de réplica daquela região não precisa enviar mensagens de fofoca com tanta frequência para os gerenciadores de outros continentes.

Essa replicação de dados com base na geolocalização ou em um chamado gráfico social é comum em aplicações distribuídas, principalmente redes sociais. As operações de consulta são mais frequentes que as de atualização, e a disponibilidade sempre vem antes da consistência. Um exemplo bastante conhecido desse tipo de organização de dados é o sistema The Associations of Objects (TAO) criado pelo Facebook (BRONSON *et al*., 2013). O *design* utilizado para a replicação no sistema TAO é o *master/slave* (primário/auxiliar), que garante que as requisições de consulta serão atendidas dentro de uma região, porém, com o risco de algumas retornarem dados obsoletos.

Os sistemas de arquivos distribuídos são aplicações que utilizam o conceito de replicação. Dois exemplos bem populares desse tipo de sistema são: Google File System (GFS) e Apache Hadoop, ambos utilizados para servir com alta disponibilidade grandes volumes de dados. O GFS utiliza o conceito de replicação passiva, mantendo cópias primárias e secundárias, ao passo

que o Hadoop pode fazer várias cópias de um dado em um mesmo servidor (GEMAYEL, 2016).

A replicação possui diversas aplicações práticas, e sua gestão possibilita alta disponibilidade, tolerância a falhas e confiabilidade, que são requisitos de muitos sistemas. Contudo, apenas usar a replicação não garante que esses requisitos sejam cumpridos, deve-se atentar ao modelo que terá o melhor desempenho na aplicação em questão. Se analisar o grande volume de dados que os sistemas computacionais devem manter atualmente, você terá certeza de que a replicação ainda será objeto de muitos estudos e experimentos.



Referências

BRONSON, N. *et al*. TAO: Facebook's Distributed Data Store for the Social Graph. *In*: USENIX ANNUAL TECHNICAL CONFERENCE, 2013, San Jose. *Proceedings* [...]. San Jose: Fairmont San Jose, 2013. p. 49–60. Disponível em: https://www.usenix.org/conference/atc13/technical-sessions/presentation/bronson. Acesso em: 24 jun. 2020.

COULOURIS, G. *et al. Sistemas distribuídos*: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013. 1064 p.

GEMAYEL, N. Analyzing Google File System and Hadoop Distributed File System. *Research Journal of Information Technology*, v. 8, n. 3, p. 66–74, 2016. Disponível em: https://scialert.net/abstract/?doi=rjit.2016.66.74. Acesso em: 24 jun. 2020.

LADIN, R. *et al.* Providing high availability using lazy replication. *ACM Transactions on Computer Systems*, New York, v. 10, n. 4, p. 360–391, Nov. 1992. Disponível em: https://dl.acm.org/doi/abs/10.1145/138873.138877. Acesso em: 24 jun. 2020.

TANENBAUM, A. S.; STEEN, M. *Sistemas distribuídos*: princípios e paradigmas. 2. ed. São Paulo: Pearson Prentice Hall, 2008. 402 p.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:

SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS